UNIVERSITÀ DEGLI STUDI DI MESSINA

DIPARTIMENTO DI INGEGNERIA

Corso di Laurea in Ingegneria Gestionale

Uso di modelli di Long Short-Term Memory per l'Anomaly Detection negli impianti industriali

Tesi di Laurea di:

Michele Ciaramidaro

Relatore:

Chiar.mo Prof.
Dario Bruneo

ANNO ACCADEMICO 2023/2024

A mio padre e mia madre,
che mi sono sempre vicino.

A mio fratello Christian e mia sorella Beatrice
che mi hanno fatto crescere.

Ai miei nonni,
che mi hanno sempre sostenuto.

Ai miei amici ,
che mi motivano a dare e cercare il meglio.

Ringraziamenti

Desidero esprimere la mia più profonda gratitudine ai miei genitori per il loro incondizionato supporto e amore, la loro guida e incoraggiamento costante sono stati fondamentali per il raggiungimento di questo traguardo.

Un ringraziamento speciale va a mio fratello Christian e mia sorella Beatrice la loro presenza nella mia vita è stata una fonte inesauribile di forza e ispirazione.

Voglio anche ringraziare i miei nonni, che con la loro saggezza e affetto mi hanno sempre sostenuto, le loro storie e il loro esempio mi hanno insegnato l'importanza della dedizione e del duro lavoro.

Un grazie di cuore ai miei zii, il loro sostegno e i loro consigli preziosi sono stati di grande aiuto lungo questo percorso la loro presenza nella mia vita ha arricchito la mia esperienza e mi ha dato la forza di continuare.

Vorrei inoltre ringraziare i miei amici, che si sono rivelati cruciali in questo percorso, facendomi ridere, distrarre e consigliandomi, nonchè ispirandomi e facendomi apprendere concetti e tecniche fuori dal mio ambito. In particolare vorrei ringraziare Ivan e Valerio, amici da sempre che nonostante le loro difficoltà ci sono stati in ogni momento, battendosi per me e con me . Vorrei ringraziare Gloria, amica e collega, che mi ha accompagnato in questo lungo percorso sin dal giorno zero, spesso salvandomi dalle situazioni accademiche più spinose. Infine vorrei ringraziare Eleonora, sei stata essenziale, questa laurea è tanto mia quanto tua .

Ringrazio il professore Dario Bruneo e il professore Fabrizio De Vita, per avermi seguito, consigliato e per avermi stimolato a dare il meglio, alimentando la mia sete di curiosità.

Indice

1 Introduzione **1**
1.1 Anomaly Detection 1
1.2 Time Series Analysis e Pattern Recognition 2
1.3 Obiettivi 2
1.4 Keras e TensorFlow 2

2 Costruzione del dataset e pulizia dei dati **4**
2.1 impianto preso in esame per lo studio 4
2.2 Sensori impiegati 4
2.3 Dataset 5
2.4 Filtro di Kalman 6
2.4.1 Definizione del problema 6
2.4.2 Forma del filtro 8
2.4.3 Applicazione del Filtro al nostro studio 9
2.5 Estrazione della stagionalià 10
2.6 Data normalization 11

3 Apprendimento automatico **12**
3.1 Artificial intelligence 12
3.2 Machine Learning 12
3.3 L'apprendimento profondo 13
3.3.1 Machine learning e Deep learning:Differenze 13
3.4 Artificial neural networks 14
3.4.1 Caratteristiche delle neural networks 14
3.5 Apprendimento delle neural networks 15
3.6 Training e valutazione di una neural network 16
3.6.1 Gradient descent 17
3.6.2 Dataset split 18
3.6.3 Adam 18
3.6.4 Categorical cross entropy 18
3.7 Overfitting 19

4 RNN e LSTM **20**
4.1 RNN 20
4.2 Vanishing gradient 20

4.3 Long Short-Term Memory 21
4.3.1 Forget gate 22
4.3.2 Input gate 23
4.3.3 Output gate 24

5 Autoencoders 26
5.1 Generalità 26
5.1.1 Encoding 26
5.1.2 Decoding 27
5.1.3 Reconstruction Loss 27
5.1.4 LSTM-Autoencoder 27

6 Anomaly Detector 28
6.1 Architettura 28
6.2 Finestre temporali 28
6.3 Reshape dei dati per l'LSTM 29
6.4 LSTM-Autoencoder impiegata 29
6.4.1 Numero di epoche 31
6.4.2 Batch size 31
6.4.3 Validation split 31
6.5 Loss di training 31
6.6 Loss distribution 32
6.7 Rilevare le anomalie 33

7 Conclusioni 34

Elenco delle figure

1 Grafici delle vibrazioni ricevute dal sensore di vibrazione nelle sue tre direzioni assiali, condizioni anomale (primi tre a sinistra), condizioni normali 5
2 Grafici dei set di addestramento e di test 6
3 Grafici dei dati di training e test filtrati con Kalman 10
4 Grafici delle decomposizioni stagionali effettuate sulla serie temporale anomala e su quella sana 11
5 Funzionamento della cella di una rete 16
6 Cella RNN e LSTM 21
7 Forget gate 22
8 Input gate 23
9 Output gate 24
10 Struttura generale di un autoencoder[7] 26
11 Grafico della funzione di loss durante l'addestramento del modello . . 32
12 Grafico del Distribution plot della funzione di loss sui dati di training . 32
13 Grafico del Distribution plot dei dati di training e test con anomalie . . 33
14 Metriche di performance 34

Capitolo 1

Introduzione

A seguito dell'avvento dell'Industria 4.0, le aziende generano una quantità senza precedenti di dati attraverso sensori IoT, macchinari intelligenti e sistemi di automazione. Questi dati, se analizzati sapientemente, possono fornire insights preziosi per migliorare l'efficienza operativa, ridurre i costi, ottimizzare i processi produttivi e aumentare la qualità dei prodotti. Il *Machine Learning* e il *Deep Learning* potendo analizzare e interpretare grandi volumi di dati, giocano un ruolo fondamentale nel fare ciò, e le loro applicazioni sono le più svariate, spaziando dal riconoscimento vocale fino alle applicazioni mediche, per citare alcune applicazioni del *Machine Learning* in ambito industriale abbiamo:la manutenzione predittiva, l'ottimizzazione della produzione, la gestione della supply chain e il controllo della qualità. Mentre il *Deep Learning* estendendo le capacità analitiche, risulta particolarmente utile quando si tratta di elaborare dati non strutturati come immagini, video e testi, le reti neurali profonde sono utilizzate, ad esempio, nella visione artificiale per il controllo qualità visivo, dove possono individuare difetti nei prodotti finiti con una precisione superiore rispetto ai metodi tradizionali.

L'integrazione delle tecnologie di machine learning (ML) e deep learning (DL) nell'Industria 4.0 sta rivoluzionando profondamente il modo in cui le aziende processano e interpretano i dati, promuovendo un approccio sempre più data-driven, questa trasformazione consente alle imprese di diventare più agili, reattive ed efficienti, permettendo loro di mantenere un vantaggio competitivo nel mercato globale. Il seguente documento esplorerà in dettaglio le applicazioni del *Machine Learning* e del *Deep Learning* nel campo dell'Anomaly Detection, evidenziando come queste tecnologie stiano plasmando il futuro del settore industriale. Tuttavia, l'adozione di ML e DL non è priva di sfide, tra queste, la gestione della complessità dei dati, la necessità di infrastrutture tecnologiche avanzate, e la formazione di personale qualificato sono alcuni degli ostacoli che le aziende devono affrontare, inoltre, le preoccupazioni riguardanti la sicurezza dei dati e la privacy sono questioni cruciali che devono essere adeguatamente gestite per garantire un'implementazione sicura e affidabile di queste tecnologie.

1.1 Anomaly Detection

L'*"Anomaly Detection"* è una tecnica usata nell'ambito del ML e del DL, che è volta a rilevare comportamenti anomali una volta identificati e stabiliti i patterns regolari dei

dati presi in esame.

1.2 Time Series Analysis e Pattern Recognition

Una *Time Series* è consiste in una serie di dati raccolti e registrati cronologicamente, nella serie temporale ogni valore è misurato a un determinato intervallo di tempo[11]. Questi dati possono essere raccolti a intervalli regolari o irregolari, a seconda delle esigenze e del contesto dello studio.

Il *Pattern Recognition* è una disciplina dell'intelligenza artificiale e dell'informatica che si occupa di identificare e classificare modelli o schemi in dati complessi. L'obiettivo è riconoscere strutture o regolarità in un insieme di dati e usarle per fare previsioni o prendere decisioni. Questa tecnica viene utilizzata in una vasta gamma di applicazioni, dai sistemi di riconoscimento vocale ai sistemi di visione artificiale, dalla biometria alla diagnostica medica.

1.3 Obiettivi

Questo studio si propone di sviluppare un modello di rete neurale ricorrente *Long Short-Term Memory* (LSTM) con autoencoder, al fine di rilevare comportamenti anomali nel funzionamento di una replica di un impianto industriale preposto al trasporto di componeneti automobilistiche. L'obiettivo è analizzare i dati raccolti dai sensori,in particolare gli accellerometri, sotto la forma di serie temporali e indentificare quei pattern che si discostano dalle condizioni operative normali dell'impianto, per realizzare ciò sono stati raccolti i dati dell'impianto di trasporto senza carico (considerati normali) e quelli con carico (anomali) e successivamente sono stati uniti in un unico dataset.

1.4 Keras e TensorFlow

Il modello di LSTM con autoencoder è stato realizzato mediante l'utilizzo di *Keras* e *TensorFlow*.

TensorFlow è una libreria open source per l'apprendimento automatico sviluppata da Google, ampiamente utilizzata per addestrare e distribuire modelli di machine learning.

Con TensorFlow, è possibile definire l'architettura di un modello, specificando il numero di livelli, il tipo di funzioni di attivazione e l'ottimizzatore da utilizzare. Inoltre, la libreria offre strumenti per addestrare il modello su vari set di dati. Uno dei principali punti di forza di TensorFlow è la sua scalabilità: può essere utilizzato su una

vasta gamma di hardware, da singole CPU a cluster distribuiti. Inoltre, offre un'interfaccia visiva che facilita il debug e il monitoraggio dell'addestramento, permettendo agli sviluppatori di diagnosticare e risolvere facilmente eventuali problemi nei loro modelli.

Google col progetto Google Brain lo sviluppò nel 2015 con licenza libera. Il nome è già abbastanza esplicativo in quanto è composto dal termine *Tensor*, che rappresenta un array multidimensionale e dalla parola *Flow*,che letteralmente significa "flusso" ma in questo contesto rappresenta il flusso di operazioni.

Keras è una rinomata libreria open source per il Deep Learning, scritta in Python. Si distingue per la sua API di alto livello che semplifica la costruzione e l'addestramento di reti neurali profonde, rendendo accessibile la creazione di modelli avanzati di machine learning anche a chi non ha una conoscenza approfondita della matematica e degli algoritmi sottostanti, è ampiamente utilizzata in una vasta gamma di applicazioni di apprendimento automatico, tra cui la image classification il natural language processing, e recommendation systems.Una delle caratteristiche distintive di Keras è la sua flessibilità e modularità. Gli sviluppatori possono combinare e personalizzare una serie di livelli pre-costruiti, come i livelli convoluzionali per il trattamento delle immagini, i livelli ricorrenti per dati sequenziali e i livelli densi per dati generici,offre inoltre la possibilità di scegliere tra diversi algoritmi di ottimizzazione, funzioni di perdita e funzioni di attivazione, consentendo agli utenti di ottimizzare i loro modelli e migliorarne le prestazioni in base alle specifiche esigenze del problema.

Capitolo 2

Costruzione del dataset e pulizia dei dati

2.1 impianto preso in esame per lo studio

Come già accennato nell'introduzione l'impianto per il quale abbiamo effettuato l'Anomaly Detection, consiste nella replica in scala di un impianto preposto al trasporto e all'assemblaggio di componenti automobilistiche. Il sistema è costituito da due motori e un set di sei nastri per il trasporto di un carrello mobile. Tale impianto ha la caratteristica di poter iniettare i guasti al suo interno, i quali sono di natura meccanica, come: generare vibrazioni esterne nella struttura dell'impianto, modificare la tensione della cinghia, aumentare l'attrito degli ingranaggi agendo sui freni del sistema e disattivare l'interruttore di prossimità del carrello.

Questa caratteristica è di primaria importanza per progettare applicazioni di intelligenza artificiale poiché consente di avere una visione chiara del sistema, specialmente quando è soggetto a stress meccanico[5].

2.2 Sensori impiegati

Per monitorare il banco di prova, sia dal punto di vista meccanico che elettrico, sono stati impiegati diversi sensori. Specificamente per la parte meccanica sono stati usati:

- Il sensore di vibrazione *VTV-122*: prodotto da *IFM electronics*, questo sensore è alimentato da una tensione continua che varia tra 9 e 32V e segna in uscita un segnale analogico che rispetta lo standard 4-20mA[5].

- Il sensore di prossimità *VL53L0X*: prodotto da *STmicroelectronics*, emette un raggio di fotoni che vengono riflessi quando incontrano un ostacolo lungo il loro percorso, il sensore può riflettere questi fotoni e calcolare la distanza. Usa il protocollo di comunicazione standard 12C[5].

- Il sensore di temperatura *TS2229*: è stato utilizzato per misurare la temperatura dei motori. È un sensore composto da una sonda flessibile particolarmente adatta per misurare la temperatura delle superfici solide. A seconda della temperatura rilevata, il sensore genera in uscita un segnale analogico utilizzando lo standard 4-20 mA[5].

- Il microfono *SPQ0410HR5H-B* : prodotto da *Mikroe*, è un microfono omnidirezionale che genera in uscita un segnale analogico comunicando utilizzando l'interfaccia Serial Peripheral Interface (SPI), UART e I2C[5].

Infine, per monitorare la parte elettrica del banco di prova, è stato utilizzato un sensore di corrente *SCT-013* in grado di misurare la corrente in modo non invasivo. Il sensore funziona come un trasformatore, generando in uscita una tensione proporzionale alla corrente che scorre in un condensatore interno[5].

Tuttavia per questa analisi sono stati considerati solo i dati del sensore di vibrazione *VTV-122* in quanto l'anomalia è stata simulata da due condizioni di lavoro differenti (con carico e senza carico).

2.3 Dataset

I dataset originali erano composti da tre features che rappresentano le tre direzioni assiali del sensore di vibrazione *VTV-122*.

Il dataset contenente i valori non anomali era composto da tre colonne e 9632614 righe, ciascun valore è stato rilevato con una frequenza di campionamento di 400 punti al secondo, mentre il dataset contenente i dati anomali era formato sempre da tre colonne e da 531370 righe, anche in questo caso la frequenza di campionamento era di 400 punti al secondo.

Per condurre questo studio si è scelto di mantenere come unica feature solo quella rappresentante l'asse z, in quanto era quella più sollecitata alle vibrazioni.

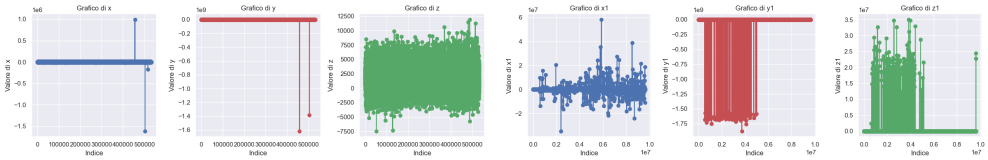
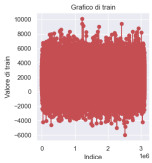
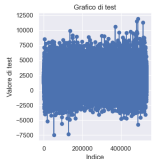


Figura 1: Grafici delle vibrazioni ricevute dal sensore di vibrazione nelle sue tre direzioni assiali, condizioni anomale (primi tre a sinistra), condizioni normali

Come si evince dai grafici in figura 1, nei dati non anomali sono presenti dei picchi, i quali sono da imputare a degli errori di set up dei sensori, per tale motivo si è scelto di escludere quelle osservazioni dal dataset, portando le righe presenti nel dataset a 3100000. Successivamente si è potuto procedere alla costruzione del *Training Set* formato dalle osservazioni originali del dataset contenente i valori non anomali meno le ultime mille osservazioni che sono state sostituite con le prime mille del dataset contenente i dati anomali, mentre il *Test Set* è formato dalle osservazioni del dataset contenente i dati anomali meno le prime mille le quali sono state sostituite con le ultime mille della serie sana.



(a) Training Set



(b) Test Set

Figura 2: Grafici dei set di addestramento e di test

2.4 Filtro di Kalman

Il Filtro di Kalman è un metodo ricorsivo ideato da Rudolf E. Kalman per stimare lo stato di un sistema dinamico a partire da una serie di misurazioni affette da rumore [8].

Il problema della progettazione del filtro rappresenta il dualismo del problema del Regolatore Lineare Quadratico.

2.4.1 Definizione del problema

Consideriamo un generico sistema MIMO: dato un sistema dinamico lineare stazionario sottoposto a del rumore di processo $v_x(t)$ e rumore di misura $v_y(t)$, si scrivono le equazioni caratteristiche come:

$$\dot{x}(t) = Ax(t) + Bu(t) + v_x(t) \tag{1}$$

$$y(t) = Cx(t) + Du(t) + v_y(t) \tag{2}$$

con

$$v(t) = \begin{bmatrix} v_x(t) \\ v_y(t) \end{bmatrix}$$

[8]

dove i rumori $v_x(t)$ e $v_y(t)$ sono incorrelati nel tempo, congiuntamente gaussiani a media nulla. Senza perdere generalità si può considerare $D = 0$ (sistema strettamente proprio). Dato il rumore, si scriva una matrice V detta matrice di covarianza come:

$$V = \begin{bmatrix} \tilde{Q} & Z \\ Z & \tilde{R} \end{bmatrix}$$

dove

$$Z = \mathrm{Cov}[v_x(t), v_y(t)], \quad \tilde{Q} = \mathrm{Var}[v_x(t)], \quad \tilde{R} = \mathrm{Var}[v_y(t)]$$

[8]

Le ipotesi aggiuntive considerate sono le seguenti:

- Covarianza non nulla sulla misura ($\tilde{R} \succ 0$): Questa condizione indica che il rumore presente nelle misurazioni è significativo e varia su ogni uscita del sistema. In pratica, significa che le misure non sono perfettamente accurate e contengono un certo livello di incertezza, rappresentato dalla covarianza $\tilde{R}$. Questa variazione nella misura rende necessario considerare tale incertezza nel processo di stima dello stato del sistema.

- Incorrelazione dei rumori di stato e di uscita ($Z = 0$): Questa ipotesi stabilisce che i rumori che influenzano lo stato del sistema e quelli che influenzano le misurazioni (uscite) sono indipendenti l'uno dall'altro. Ciò significa che la variazione nel sistema interno non è legata o correlata alla variazione delle misurazioni esterne. Questa separazione dei rumori è importante perché permette di trattare separatamente le fonti di incertezza che influenzano lo stato interno del sistema e quelle che influenzano le misurazioni esterne.

- Stato iniziale modellato come variabile casuale gaussiana:

$$x_0 = x(0)$$

$$\mathrm{E}[x(0)] = \bar{x}_0$$

$$\mathrm{E}[(x_0 - \bar{x}_0)(x_0 - \bar{x}_0)'] = \tilde{P} \succeq 0$$

  dove x_0 rappresenta lo stato iniziale del sistema al tempo $t = 0$, $\bar{x}_0$ è il valore atteso (media) dello stato iniziale, indicato come $\mathrm{E}[x(0)]$, e $\tilde{P}$ è la covarianza dello stato iniziale x_0 rispetto alla sua media $\bar{x}_0$. Questo indica quanto lo stato iniziale può variare intorno al suo valore atteso $\bar{x}_0$.

e inoltre i rumori e lo stato sono incorrelati, ovvero:

$$\mathrm{E}[x_0 v'] = 0$$

Osservatore dello stato

Si consideri a questo punto l'osservatore

$$\frac{\mathrm{d}\hat{x}(t)}{\mathrm{d}t} = \dot{\hat{x}}(t) = A\hat{x}(t) + Bu(t) + L(t)[y(t) - C\hat{x}(t)]$$

Da qui, tramite passaggi algebrici per sostituzione, è possibile scrivere la dinamica dell'errore nel seguente modo:

$$\dot{e}(t) = \dot{x}(t) - \dot{\hat{x}}(t)$$

$$\dot{e}(t) = A_c(t)e(t) + B_c(t)v(t)$$

dove si definiscono

$$A_c(t) = A - L(t)C$$

$$B_c(t) = I - L(t)$$

Scrivendo ora

$$\overline{e}(t) = \mathrm{E}[e(t)]$$

$$\overline{\dot{e}}(t) = A_c(t)\overline{e}(t) + B_c\mathrm{E}[v(t)] = A_c(t)\overline{e}(t)$$

Il valore atteso dell'errore segue una dinamica in evoluzione libera non forzata, come nel caso di un sistema autonomo. Si definisca la matrice di covarianza dell'errore[8]

$$\tilde{P}(t) = \mathrm{E}[e(t)e(t)']$$

$$\tilde{P}(0) = \tilde{P}_0$$

L'obiettivo di ottimizzazione è quindi trovare $L(t)$ che minimizza la cifra di merito[8]

$$\min_{L(t)} \gamma'\tilde{P}(t)\gamma = \min_{L(t)} \|\gamma\|^2_{\tilde{P}(t)}$$

con γ vettore di dimensioni opportune.

2.4.2 Forma del filtro

Si va a dimostrare che il guadagno $L(t)$ soluzione del problema di ottimizzazione ha la forma

$$L(t) = \tilde{P}(t)C'\tilde{R}^{-1}$$

dove $\tilde{P}(t)$ è soluzione per l'equazione di Riccati scritta nella forma

$$\dot{\tilde{P}}(t) = A\tilde{P}(t) + \tilde{P}(t)A' + \tilde{Q} - \tilde{P}(t)C'\tilde{R}^{-1}C\tilde{P}(t)$$

con condizione iniziale

$$\tilde{P}(0) = \tilde{P}_0$$

Ci troviamo nel caso duale del problema di ottimizzazione per il controllo LQ.[8]

2.4.3 Applicazione del Filtro al nostro studio

Come detto in precedenza i dati che compongono il dataset, sono stati rilevati da dei sensori di vibrazione, tali sensori sono soggetti, in una certa misura, ad un errore (o rumore) di misurazione, ed è qui che trova impiego il Filtro di Kalman, ovvero per restituirci dei dati di training e di test ancora più puliti ed accurati per l'addestramento e la valutazione della rete .

Di seguito è riportato il codice impiegato per eseguire il filtraggio di Kalman del training set, il codice impiegato per il filtraggio dei dati di test è omesso in quanto identico.

```
# Dati di misura (simulati)
n_samples = len(train)  # serie temporale dei dati di misura
measurements = train.values  # Convertiamo i dati di misura in un
    array

# Parametri del filtro di Kalman
A = 1.0  # Matrice di transizione di stato
H = 1.0  # Matrice di osservazione
Q = 0.01  # Covarianza del rumore di processo
R = 0.1  # Covarianza del rumore di misura

# Inizializzazione
x_hat = measurements[0]  # Stima iniziale dello stato
P = 1.0  # Covarianza iniziale dell'errore di stima

# Filtraggio di Kalman
filtered_state = []
for j in measurements:
    # Predizione
    x_hat_minus = A * x_hat
    P_minus = A * P * A + Q

    # Aggiornamento
    K = P_minus * H / (H * P_minus * H + R)
    x_hat = x_hat_minus + K * (j - H * x_hat_minus)
    P = (1 - K * H) * P_minus
```

```
filtered_state.append(x_hat)
```

Listing 1: Implementazione del filtro di Kalman in Python

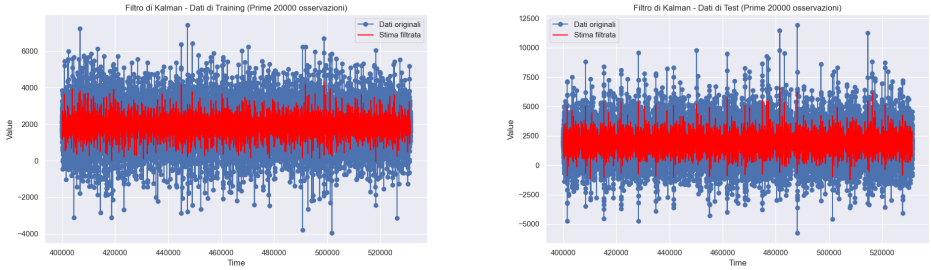


Figura 3: Grafici dei dati di training e test filtrati con Kalman

2.5 Estrazione della stagionalià

Una serie temporale, può essere scomposta nelle sue componenti costituenti tramite una tecnica statistica chiamata *Decomposizione Stagionale*, tali componenti sono :

- Trend: si riferisce ai cambiamenti a lungo termine della serie che si osservano nel tempo, inoltre rappresenta la direzione generale in cui si sta muovendo la serie.
- Stagionalità: essa si riferisce ai pattern ciclici osservati nel periodo di tempo prestabilito.
- Residui: rappresenta la variazione casuale che rimane dopo che le componenti trend e stagionalità sono state prese in considerazione. Rappresenta il rumore o l'errore nei dati che non può essere spiegato dal trend o dalla stagionalità.

In generale la *Decomposizione Stagionale* è usata per ottenere insights sulle dinamiche nascoste nei dati della serie.

Nel nostro studio dunque, dopo aver filtrato i dati tramite Kalman, abbiamo applicato tale decomposizione tramite il modulo python "statsmodels.tsa.seasonal" della libreria *"statsmodels"*, il quale offre la funzione *"seasonaldecompose"* che ha permesso la decomposizione della serie nelle componenti sopra citate.

Delle tre componenti fondamentali è stato deciso di aggiungere al dataset solo la stagionalità, in quanto i trend della serie temporale anomala e sana erano simili, e per determinare le anomalie era di interesse determinare appunto le caratteristiche che potessero differenziare le due, permettendo inoltre di avere un dataset più leggero e quindi tempi di calcolo più rapidi.

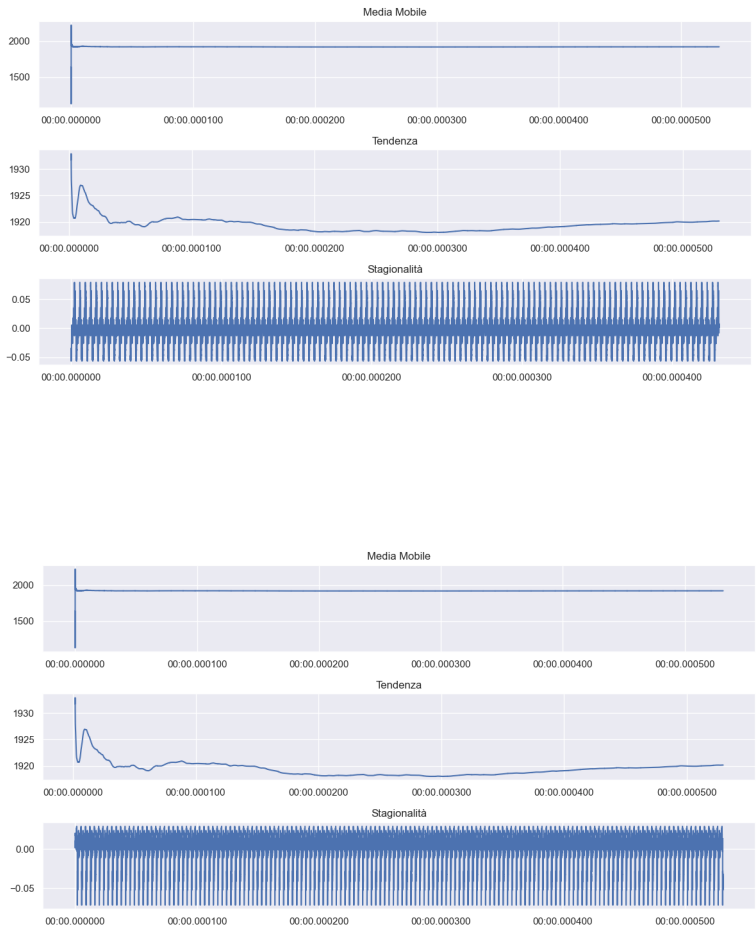


Figura 4: Grafici delle decomposizioni stagionali effettuate sulla serie temporale anomala e su quella sana

2.6 Data normalization

Infine prima di essere dati in ingresso alla rete, i dati vengono normalizzati tramite il modulo "sklearn.preprocessing" della libreria "scikit-learn" in Python, usando la classe *MinMaxScaler*, la quale scala ogni caratteristica/variabile indipendente presente nei dati in modo che rientrino in un intervallo specificato tra 0 e 1.

Dopo aver compiuto, anche questa operazione, i dataset di training e di test, sono finalmente pronti per essere usati per allenare ed addestrare la rete.

Capitolo 3

Apprendimento automatico

3.1 Artificial intelligence

L'intelligenza artificiale (IA) rappresenta un campo della scienza informatica che si dedica alla creazione di sistemi capaci di eseguire compiti che normalmente richiederebbero l'intelligenza umana. Questi compiti comprendono l'apprendimento, il ragionamento, la percezione, la comprensione del linguaggio naturale e la capacità di prendere decisioni autonome. L'IA ha radici storiche significative: nel 1956, l'informatico John McCarthy coniò il termine per la prima volta. Tuttavia, già nel 1950, Alan Turing propose l'idea di macchine in grado di comunicare con gli esseri umani senza rivelare la loro non umanità. Questo concetto segnò un inizio fondamentale per l'IA. Inoltre, Marvin Minsky, un altro pioniere dell'informatica, ha descritto l'intelligenza artificiale in termini di modelli computazionali che simulano capacità cognitive umane.

Negli anni '60 La ricerca in IA ottenne significativi finanziamenti, sviluppando i primi sistemi esperti e linguaggi di programmazione come LISP. Tuttavia, ostacoli tecnici e limitazioni hardware portarono a delusioni, poiché gli algoritmi non potevano gestire problemi complessi del mondo reale. Tuttavia negli anni '80, l'attenzione si spostò sui sistemi esperti, software progettati per risolvere problemi specifici. Questo periodo vide un rinnovato interesse e finanziamenti, con applicazioni pratiche in diagnosi medica e finanza, finchè le limitazioni dei sistemi esperti portarono a un nuovo periodo di scetticismo. Tuttavia, negli anni '90, progressi nel machine learning, come gli alberi decisionali e le reti neurali artificiali, segnarono un rinascimento per l'IA infatti l'ultimo decennio ha visto una crescita esplosiva nell'IA grazie ai progressi nel deep learning. La disponibilità di grandi quantità di dati e la potenza delle GPU hanno permesso di addestrare modelli sofisticati, con applicazioni di successo in riconoscimento vocale, visione artificiale e veicoli autonomi.

3.2 Machine Learning

Il Machine Learning (ML) è un campo dell'intelligenza artificiale dove gli algoritmi e le tecniche consentono ai computer di migliorare le loro prestazioni mediante l'apprendimento dai dati, a differenza dei tradizionali programmi informatici che sono esplicitamente programmati per svolgere compiti specifici, i sistemi di ML sono progettati per identificare pattern nei dati e trarre inferenze da essi, questo processo

permette loro di adattarsi e migliorare autonomamente nel tempo, senza bisogno di essere riscritti per ciascuna nuova situazione.

Il ML può essere classificato in diverse categorie principali, basate sul tipo di apprendimento e sull'approccio utilizzato:

- **Apprendimento supervisionato (Supervised learning)**: L'apprendimento supervisionato è uno dei paradigmi più comuni nel machine learning, dove un modello viene addestrato utilizzando un dataset di input e output noti o *labelled*. Questo tipo di apprendimento è suddiviso in due principali categorie: classificazione e regressione..

- **Apprendimento non supervisionato (Unsupervised learning)**: L'apprendimento non supervisionato viene utilizzato quando i dati di addestramento non sono etichettati. L'obiettivo è scoprire strutture o pattern nascosti nei dati. Un esempio di applicazione è l'*anomaly detection*, utilizzato per riconoscere comportamenti inusuali, come transazioni fraudolente con carta di credito.

- **Apprendimento semi-supervisionato (Semisupervised learning)**: Questo metodo è una combinazione dei due approcci precedenti. L'algoritmo riceve sia dati etichettati (con risposte corrette) che non etichettati. In questo modo, può sfruttare l'abbondanza di dati non etichettati per migliorare l'apprendimento, pur avendo accesso a un numero limitato di esempi con soluzioni corrette.

3.3 L'apprendimento profondo

Il *deep learning* (DL) è una branca del machine learning in cui il termine "deep" indica l'uso di modelli che operano su multiple strati di rappresentazione dell'informazione [3]. Questo approccio è anche denominato apprendimento stratificato delle rappresentazioni o apprendimento gerarchico delle rappresentazioni.

3.3.1 Machine learning e Deep learning:Differenze

Machine Learning (ML) e *Deep Learning* (DL) differiscono nelle modalità di addestramento di ciascun algoritmo. Nel classico approccio di ML, noto anche come "shallow learning", l'accento è posto sull'intervento umano per definire e raffinare le caratteristiche rilevanti nei dati. Questo approccio richiede un'ingegnerizzazione delle feature dettagliata e manuale per addestrare modelli efficaci. In contrasto, il Deep Learning automatizza gran parte del processo di estrazione delle caratteristiche, permettendo l'uso di dataset di grandi dimensioni. Questo consente ai modelli di DL di apprendere rappresentazioni complesse e astratte direttamente dai dati grezzi, senza la necessità di

un'intervento umano estensivo nella fase di progettazione delle caratteristiche. Questa differenza metodologica si riflette anche nell'architettura delle reti neurali utilizzate in ciascun approccio. Mentre gli algoritmi di ML si basano su reti neurali superficiali composte da uno o due strati di neuroni, i modelli di DL possono presentare centinaia o addirittura migliaia di strati successivi. Questa profondità architetturale consente alle reti neurali profonde di catturare e modellare relazioni complesse nei dati, rendendole ideali per problemi che richiedono un alto livello di astrazione e elaborazione dei dati non strutturati, come il riconoscimento delle immagini, il riconoscimento del linguaggio naturale e la guida autonoma.

3.4 Artificial neural networks

I modelli di *artificial neural networks* (ANN) sono sistemi informatici, ispirati dai sistemi nervosi biologici, la funzione e la conformazione[6].

Una ANN è costituita da numerose formazioni di preparazione, simili ai neuroni biologici, spesso chiamate nodi o processori. Ogni nodo, quando attivato, trasmette un segnale alle unità collegate. Questi punti di connessione agiscono come filtri, trasformando il segnale in un impulso "eccitatorio" o "inibitorio". Essi assegnano un "peso" al segnale, determinando la sua importanza nell'elaborazione complessiva. A causa di questa funzione specifica, le connessioni tra i nodi sono chiamate *sinapsi*, in analogia con le sinapsi del sistema nervoso umano, riflettendo il loro ruolo critico nella trasmissione e modulazione dei segnali all'interno della rete. Analogamente al sistema nervoso umano, che è caratterizzato da una complessa rete di connessioni sinaptiche tra neuroni, anche i nodi in una rete neurale artificiale sono interconnessi e possono comunicare tra loro. Questi nodi possono essere classificati come unità di ingresso, di uscita o nascoste (*hidden*), a seconda della loro posizione e funzione nella rete. Le unità di ingresso ricevono i dati iniziali, le unità di uscita producono il risultato finale, mentre le unità nascoste, organizzate in vari *layers*, elaborano le informazioni tra l'ingresso e l'uscita, consentendo alla rete di apprendere e riconoscere pattern complessi.

3.4.1 Caratteristiche delle neural networks

Ogni rete neurale artificiale possiede delle caratteristiche uniche, ma alcune proprietà sono comuni a tutte:

- **Robustezza**: Questa proprietà consente alla rete di funzionare correttamente anche in presenza di rumore nel segnale o di interruzioni nelle connessioni. Il disturbo dei segnali può compromettere il processo di apprendimento della rete, aumentando la percentuale di errore e diminuendo le prestazioni complessive

dell'ANN. Ad esempio, in un problema di riconoscimento, una rete poco robusta potrebbe confondere l'immagine di un'auto con quella di una mela. Questo evidenzia l'importanza cruciale della robustezza per l'affidabilità della rete.

- **Flessibilità**: Un modello neurale non è limitato a un singolo problema specifico, ma può essere applicato a una vasta gamma di compiti. Ciò non significa che una rete possa risolvere qualsiasi problema, ma che può imparare le caratteristiche rilevanti del problema attraverso l'esperienza, senza la necessità di conoscere a priori tutte le proprietà del dominio.

- **Generalizzazione**: La capacità di generalizzare permette a una rete, addestrata su un set di dati limitato, di rispondere in modo appropriato a nuovi stimoli simili a quelli precedentemente incontrati. Questa proprietà è essenziale per i modelli applicati in campi dove non è possibile avere una raccolta esaustiva di dati per l'addestramento. La generalizzazione garantisce che la rete possa adattarsi e operare efficacemente in situazioni non previste durante la fase di addestramento.

3.5 Apprendimento delle neural networks

Le reti neurali artificiali (ANN) sono costituite da un insieme di *layer*, ognuno dei quali può ospitare da uno fino a centinaia di migliaia di neuroni. Ogni *layer* funge da filtro all'interno del modello, contribuendo a una progressiva distillazione dei dati. Il compito di ciascun *layer* è quello di elaborare i dati in ingresso per estrarre una rappresentazione sempre più significativa del problema, semplificando così il processo di apprendimento. Durante questo processo di filtraggio, le informazioni estratte vengono codificate nei *pesi* del *layer*, conosciuti anche come *parametri*, che sono rappresentati da insiemi numerici.

Dal punto di vista tecnico, la trasformazione dei dati operata dai *layer* è governata dai *pesi*.

Gli ingressi di ogni neurone vengono prima amplificati o attenuati attraverso le terminazioni sinaptiche, modulate dai pesi sinaptici w_i. Successivamente, questi ingressi vengono sommati algebricamente per formare l'input netto al neurone.

$$A_i = \sum_{i}^{N} w_i x_i \tag{3}$$

Se il totale ponderato degli input non supera la soglia fissata dal bias (θ), il neurone resta inattivo e non processa alcuna informazione.

Quando il neurone decide di rispondere, prende il valore del potenziale di attivazione A e lo passa attraverso una funzione di attivazione $\Phi(A)$, che determina l'effettiva

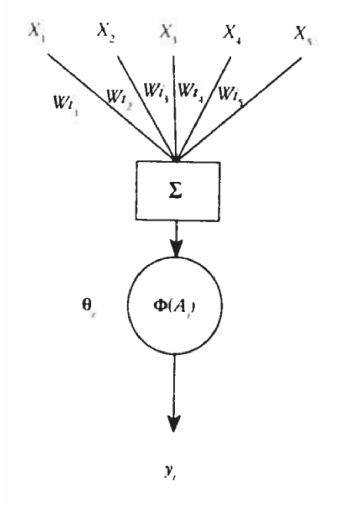


Figura 5: Funzionamento della cella di una rete [6]

uscita del neurone. Questa funzione è ciò che permette al neurone di trasformare il segnale grezzo in una risposta significativa, regolando così il comportamento della rete neurale.

$$y_i = \Phi(A_i) = \Phi(\sum_{i}^{N} w_i x_i - \theta) \tag{4}$$

Gli ingressi della rete sono dati multidimensionali detti *tensori*. Un tensore è un concetto matematico generale che estende il concetto di vettore e matrice a un numero arbitrario di dimensioni. In altre parole, mentre un vettore è un array unidimensionale e una matrice è un array bidimensionale, un tensore può avere un numero qualsiasi di dimensioni. [3].

Quindi, l'output di un layer è rappresentato come

$$y = \Phi(\mathrm{dot}(x, W) + b) \tag{5}$$

dove l'operatore *dot* esegue il prodotto tra tensori, x sono gli input, e W e b sono due tensori che rappresentano gli attributi del kernel e del bias del layer. Questi tensori sono inizializzati in modo casuale.

3.6 Training e valutazione di una neural network

La valutazione di un modello di intelligenza artificiale consiste nel confrontare l'output prodotto dalla rete con il risultato desiderato. Se i due non coincidono, i pesi della rete vengono aggiornati. Questo processo è noto come *training* e si svolge attraverso il *training loop*, che comprende diverse fasi:

- Inizialmente, una porzione dei dati viene estratta dal dataset per l'addestramento della rete, insieme ai relativi *targets*, ossia le risposte attese. La parte rimanente è destinata al *test set* per la fase di *evaluation*, che avviene dopo l'addestramento.
- Per ottenere le predizioni, i nuovi dati vengono valutati sul modello.
- Viene calcolata la *loss*, che quantifica la differenza tra le risposte prodotte dalla rete e i targets attesi.
- I pesi vengono aggiornati utilizzando un algoritmo di ottimizzazione che mira a ridurre la *loss*.

Una funzione di *loss* bassa indica che il modello è in grado di mappare efficacemente gli input sui *targets* corretti, suggerendo un funzionamento efficiente della rete.

L'algoritmo di *backpropagation* consente l'aggiornamento dei pesi. Basato sul metodo del *gradient descent*, aggiorna i pesi di ciascun layer in funzione della differenza calcolata tra la predizione della rete e il risultato desiderato.

3.6.1 Gradient descent

Il *gradient descent* (GD) è un algoritmo fondamentale nell'ambito dell'ottimizzazione utilizzato per minimizzare la di perdita [12]. L'idea alla base del gradient descent è piuttosto intuitiva: per trovare il minimo di una funzione, ci si sposta iterativamente nella direzione opposta al gradiente della funzione stessa. Il gradiente di una funzione è un vettore che punta nella direzione di massima crescita della funzione stessa. Pertanto, muovendosi nella direzione opposta del gradiente, si spera di trovare un punto di minimo locale o globale della funzione. Geometricamente, il gradiente indica la direzione in cui una funzione cresce più rapidamente; quindi, aggiornare i pesi nella direzione opposta al gradiente della *loss* permette di minimizzare questa funzione.

Il *learning rate* è uno dei parametri chiave nell'addestramento dei modelli di machine learning e deep learning che utilizzano tecniche basate sull'ottimizzazione del gradient descent. Questo parametro determina quanto i parametri del modello devono essere modificati in risposta al gradiente calcolato durante l'aggiornamento dei pesi, controlla la dimensione dei passi che l'algoritmo di ottimizzazione compie lungo la direzione del gradiente. Un learning rate troppo piccolo può rallentare il processo di convergenza, richiedendo più iterazioni per raggiungere il minimo globale o locale della funzione di costo. D'altra parte, un learning rate troppo grande può causare oscillazioni intorno al minimo o addirittura una divergenza, portando il modello a non convergere correttamente[9].

Invece di calcolare il gradiente della funzione di perdita sull'intero dataset in una sola volta, questo viene fatto su piccoli gruppi di dati chiamati *batches*. Quindi, alla fine di ogni*batch* i pesi vengono aggiornati, anziché alla fine di ogni *epoch*, che rappresenta un ciclo completo di allenamento su tutto il set di dati.

3.6.2 Dataset split

Il processo di apprendimento di un modello di rete neurale viene ulteriormente ottimizzato attraverso la suddivisione dei dati in tre gruppi distinti, utilizzati in diverse fasi:

- il training set, costituito dai dati utilizzati durante la fase di addestramento della rete
- il validation set, composto da quella porzione di dataset impiegata per valutare l'efficacia degli iperparametri adottati
- il test set, che rappresenta l'insieme di esempi forniti alla rete durante la fase di valutazione per verificare le prestazioni del sistema e la sua capacità di generalizzare il problema

3.6.3 Adam

Adam (Adaptive Moment Estimation) è un algoritmo di ottimizzazione molto popolare e efficace utilizzato nel training dei modelli di deep learning. È una combinazione di due concetti chiave: il metodo del momento, che mira a accelerare la convergenza aggiungendo un momento alla discesa del gradiente, e RMSprop (Root Mean Square Propagation), che adatta il learning rate per ciascun parametro in modo automatico.

3.6.4 Categorical cross entropy

La *categorical cross entropy* è una funzione di loss ampiamente utilizzata nei problemi di classificazione multiclasse, soprattutto quando si lavora con reti neurali che producono output softmax. È una misura della discrepanza tra la distribuzione delle probabilità predetta dal modello e la distribuzione reale delle classi. Essa misura la distanza tra due distribuzioni di probabilità, rendendo minima la *loss*. La funzione *softmax* è utilizzata per trasformare un vettore di valori in un nuovo vettore che rappresenta una distribuzione di probabilità normalizzata. Questo significa che ogni valore nel vettore di output è compreso tra 0 e 1, e la somma di tutti i valori nel vettore risultante è pari a 1..

3.7 Overfitting

L'*overfitting* è un problema frequente nel machine learning e nelle reti neurali, che si manifesta quando un modello apprende eccessivamente dai dati di addestramento, compromettendo così la sua capacità di generalizzare su nuovi dati non visti [4]. Tuttavia, l'assenza di *overfitting* potrebbe indicare che il modello non sta imparando adeguatamente. Questo può essere causato da diversi fattori, come un numero inadeguato di parametri nella rete, un dataset che non rappresenta bene il problema, un *learning rate* troppo basso o un numero di *epochs* insufficiente. Di conseguenza, l'*overfitting* deve essere tenuto sotto controllo, ma solo dopo molte epoche di addestramento.

Capitolo 4

RNN e LSTM

4.1 RNN

Le *recurrent neural networks* (RNN) sono un tipo di architettura neurale progettata per gestire dati sequenziali, dove l'ordine e la struttura temporale sono fondamentali. A differenza delle reti neurali feedforward, che trasmettono l'informazione solo in avanti attraverso i vari strati, le RNN sono in grado di mantenere uno stato interno (o memoria) che consente loro di elaborare sequenze di dati di lunghezza variabile. I dati presenti nelle serie temporali non possono essere efficacemente elaborati con approcci *fully connected* o convoluzionali perchè trattano ciascun dato come un'entità indipendente quindi non considerano l'ordine o la struttura temporale dei dati, mentre il secondo approccio è progettato per rilevare pattern spaziali locali, ma non sono naturalmente adatte per catturare dipendenze temporali lunghe o a lungo raggio. Nelle serie temporali l'ordine e la causalità sono hanno particolare rilevanza. Per tale motivo sono state progettate le RNN.

Le RNN si distinguono per via del concetto di memoria, mantenendo uno stato relativo agli input precedenti che influenza le uscite future. Per realizzare ciò, una RNN incorpora un *loop* interno.

Immaginate una rete neurale ricorrente, una RNN, che, quando passa a una nuova sequenza di dati, come saltare a un altro batch, resetta il suo stato interno. Questo significa che, in ingresso, dobbiamo specificare non solo i timestep e le features, ma anche la dimensione del batch. In uscita, invece, la RNN può restituire due tipi di risultati: può tornare l'intera sequenza di output successivi per ogni timestep, generando così tensori tridimensionali simili all'input, oppure può restituire solo l'ultimo output per ciascuna sequenza di ingresso.

Ora, se vogliamo mettere più livelli ricorrenti in cascata, dobbiamo far sì che la rete restituisca il primo tipo di uscita, l'intera sequenza di output. Perché? Perché questo output diventa l'ingresso di un altro livello ricorrente e, quindi, deve mantenere la struttura tridimensionale. Quindi, la chiave è garantire che l'informazione fluisca correttamente tra i livelli, mantenendo l'integrità dei dati temporali.

4.2 Vanishing gradient

Le reti neurali ricorrenti (RNN) sono progettate per mantenere informazioni sugli input dei timesteps precedenti, ma affrontano una sfida nota come il fenomeno del *vanishing*

gradient che si verifica durante l'addestramento delle RNN e nella sua variante come LSTM. Esso si manifesta quando i gradienti calcolati durante la retropropagazione diventano estremamente piccoli man mano che si propagano all'indietro attraverso i layer della rete durante il processo di apprendimento. Questo rende difficile aggiornare efficacemente i pesi dei layer più lontani dall'output della rete [3].

Una soluzione efficace a questo problema è l'implementazione di una *residual connection*, che agisce come una scorciatoia intorno ai blocchi che introducono disturbo. Questa scorciatoia consente alle informazioni sull'errore del gradiente dai livelli precedenti di propagarsi senza essere sopraffatte dal rumore [3].

4.3 Long Short-Term Memory

Le LSTM rappresentano una significativa evoluzione delle reti neurali ricorrenti, fornendo una soluzione efficace per l'apprendimento di sequenze temporali complesse e per la gestione delle dipendenze a lungo termine nei dati, catturando la correlazione tra dati sequenziali su periodi estesi. Queste reti sono particolarmente adatte per applicazioni come la previsione di serie temporali e lo speech recognition .

I tre elementi principali che caratterizzano l'output di una LSTM sono: lo stato della cella, rappresentante la memoria a lungo termine della rete; lo stato nascosto precedente, il quale indica l'uscita generata allo stato temporale precedente; e i dati forniti come input nello stato temporale corrente.

La struttura di una LSTM è un'evoluzione delle reti neurali ricorrenti standard, caratterizzata da un meccanismo interno composto da un loop o una catena di moduli ripetuti [10]. Questi moduli sono costruiti attorno a tre *gates* fondamentali: il gate di dimenticanza (*Forget gate*), il gate di input (*Input gate*) e il gate di output (*Output gate*), ciascuno dei quali regola il flusso di informazioni attraverso la sequenza in modo controllato e graduale.

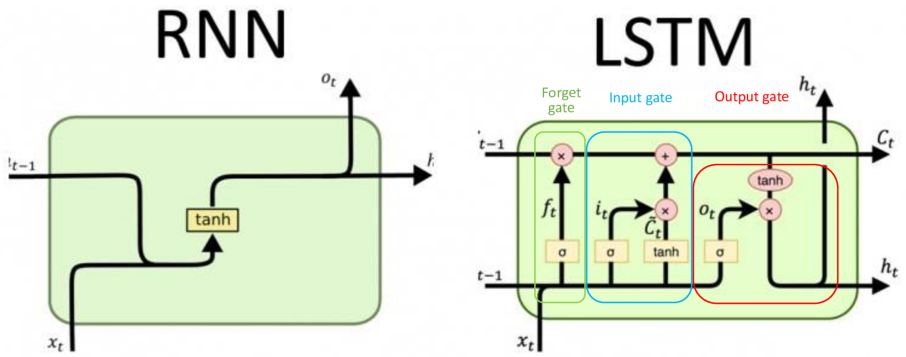


Figura 6: Confronto RNN e LSTM[10]

Nell'immagine 6, x_t indica l'input al timestep t, h_t rappresenta l'output corrispondente, C_t è lo stato della cella, f_t è il gate di dimenticanza (*forget gate*), i_t è il gate di input (*input gate*), o_t è il gate di output (*output gate*), e $\hat{C}_t$ rappresenta lo stato interno della cella.

Quindi, durante l'elaborazione sequenziale dei dati, x_t è l'input attuale, da cui vengono calcolati i vari gates e lo stato interno della cella. Questi gates (f_t, i_t, o_t) regolano come le informazioni vengono dimenticate, aggiunte e usate per produrre l'output h_t, mentre C_t rappresenta la memoria a lungo termine della LSTM.

4.3.1 Forget gate

In questo stadio, tenuto conto dello stato precedente e dei nuovi dati di input, si vanno a determinare quali componenti siano importanti o possano essere rimossi dalla cella di memoria.

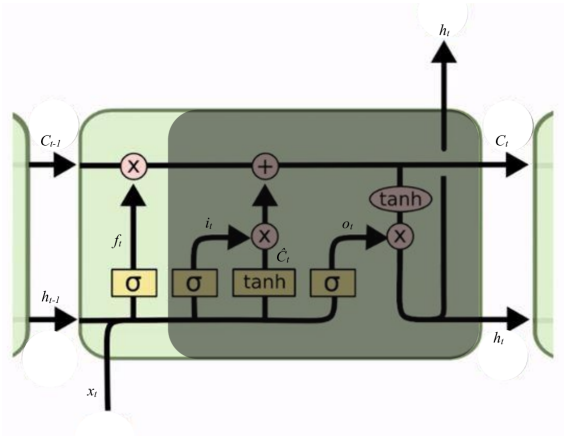


Figura 7: forget gate [10]

Questo processo avviene tramite l'uso della funzione di attivazione *sigmoid*, che riceve come input le uscite della cell precedente (h_{t-1}) e i nuovi parametri (X_t). La funzione *sigmoid* restituisce un valore tra 0 e 1 per ciascun elemento dello stato della cella precedente (C_{t-1}). Valori prossimi a 0 suggeriscono che l'informazione non è rilevante, mentre valori vicini a 1 indicano informazioni importanti [1].

$$f_t = \sigma(W_f \cdot [h_{t-1}, X_t] + b_f) \tag{6}$$

Gli output di questo gate sono moltiplicati con ogni valore dello stato precedente della cella, così le informazioni non rilevanti sono ridotte con un fattore vicino a 0, minimizzando la loro influenza nei passaggi successivi.

4.3.2 Input gate

Successivamente vi è la fase detta di *input*, dove si determinano i nuovi parametri da memorizzare nello stato della cella. Questo avviene in base allo stato nascosto antecedente e ai dati attuali in ingresso.

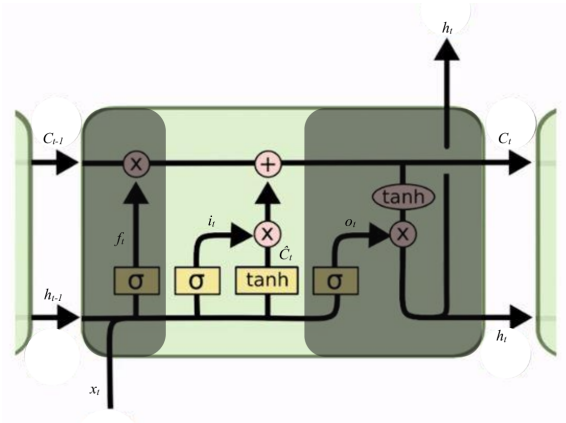


Figura 8: Schema input gate [10]

Questo processo si divide in due fasi principali: la prima fase è gestita dall'input gate, che utilizza una funzione di attivazione *sigmoid* la quale decide quali informazioni devono essere integrate nello stato della cella. La formula per calcolare l'output di questo gate è:

$$i_t = \sigma(W_i \cdot [h_{t-1}, X_t] + b_i) \tag{7}$$

Dove σ rappresenta la funzione *sigmoid*, W_i è il peso associato all'input gate, $[h_{t-1}, X_t]$ è il vettore di input che combina l'uscita precedente della cella e l'attuale input, e b_i è il bias.

La seconda fase coinvolge la creazione di un vettore di valori candidati ($\hat{C}_t$) che saranno potenzialmente aggiunti allo stato della cella. Per fare questo, si utilizza una funzione di attivazione *tanh*, che genera valori nell'intervallo da -1 a 1. Questo intervallo consente di gestire e limitare l'impatto di specifici valori negativi sullo stato della cella. La formula per questo vettore di valori è:

$$\hat{C}t = tanh(WC \cdot [h_{t-1}, X_t] + b_C) \tag{8}$$

Qui, $tanh$ è la funzione di attivazione *tanh*, W_C è il peso associato alla generazione del valore candidato, e b_C è il bias.

Infine, per aggiornare lo stato della cella, si combinano i risultati delle due fasi. Si moltiplicano l'uscita dell'input gate (i_t) con i valori candidati ($\hat{C}t$), e questo prodotto

viene aggiunto alla cella di stato precedente, pesata dal forget gate (ft). L'equazione per l'aggiornamento dello stato della cella è:

$$C_t = i_t \cdot \hat{C}t + ft \cdot C_{t-1} \tag{9}$$

In questa espressione, f_t rappresenta il forget gate, che decide quali parti dello stato della cella precedente (C_{t-1}) devono essere mantenute, e $i_t \cdot \hat{C}_t$ rappresenta la parte dello stato che viene aggiornata con le nuove informazioni.

4.3.3 Output gate

Dopo aver aggiornato lo stato della cella (C_t), il passo successivo consiste nel calcolare lo stato nascosto finale (h_t) della rete LSTM, questo stato nascosto è ottenuto combinando lo stato della cella aggiornato, lo stato nascosto precedente e i nuovi dati in ingresso.

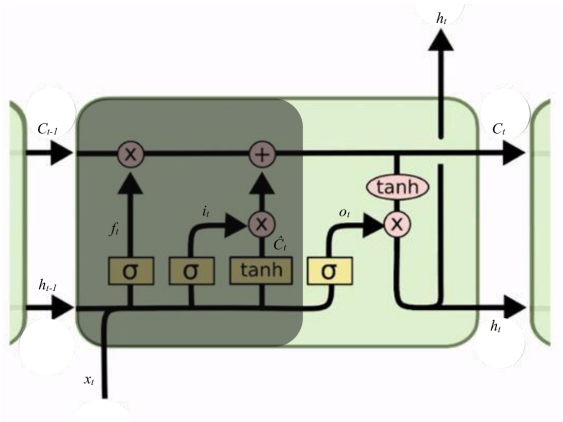


Figura 9: Schema output gate [10]

L'ultimo gate formato da un'attivazione sigmoid, produce uscite che oscillano tra 0 e 1. Gli output precedenti servono da input in questa fase nel calcolo del nuovo stato nascosto finale.

$$o_t = \sigma(W_o \cdot [h_{t-1}, X_t] + b_o) \tag{10}$$

$$h_t = o_t \times tanh(C_t) \tag{11}$$

La fase finale di una rete LSTM utilizza la funzione tangente iperbolica (tanh) per regolare l'uscita, garantendo che il nuovo stato nascosto h_t sia compreso tra -1 e 1. Questa operazione è cruciale per mantenere la stabilità dei valori e standardizzare l'output.

Successivamente, l'output h_t viene moltiplicato elemento per elemento con le uscite dell'output gate o_t, il quale determina quali informazioni conservare e trasmettere alla successiva fase di elaborazione. Questa operazione è fondamentale per regolare l'importanza delle informazioni mantenute nel nuovo stato nascosto finale.

Prima di procedere ulteriormente, l'ultimo passaggio di ogni blocco di LSTM si occupa di generare questo nuovo stato nascosto, che sarà poi trasmesso come input alla cella successiva per continuare il processo di elaborazione delle sequenze temporali.

Capitolo 5

Autoencoders

5.1 Generalità

Un autoencoder è un tipo di rete neurale che si compone di due parti essenziali: un encoder e un decoder. L'encoder, in modo simile a un esperto decodificatore di lingue, traduce l'input complesso in una rappresentazione compatta e significativa chiamata codice. Questo codice funge da chiave crittografica, catturando gli aspetti più essenziali dell'input originale. D'altra parte, il decoder agisce come un maestro linguistico, tentando di tradurre il codice ritrovato nell'input originale, sebbene con una precisione approssimativa.[7]

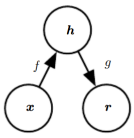


Figura 10: Struttura generale di un autoencoder[7]

Questo processo, si concentra sull'identificare e replicare solo gli elementi più rilevanti dei dati di addestramento. Questo rende gli autoencoder non solo strumenti di apprendimento delle caratteristiche, ma anche alleati nella riduzione della complessità dimensionale.

Al giorno d'oggi, gli autoencoder hanno abbracciato il caos probabilistico, sfruttando mappe che disegnano percorsi di probabilità per generare nuovi dati. Questa evoluzione ha integrato la loro applicazione nella modellazione generativa, rendendoli cardinali nel campo dell'analisi dei dati.

5.1.1 Encoding

Nell'operazione di encoding, i dati di input x sono un vettore di dimensione m ($x \in R^m$) che viene mappato in una rappresentazione in uno strato bottleneck a bassa dimensionalità h, dopo aver rimosso eventuali caratteristiche non significative[13], come mostrato nell'Equazione :

$$h = f_1(w_i x + b_i)$$

dove w_i è la matrice dei pesi, b_i è il bias e f_1 è una funzione di attivazione.

5.1.2 Decoding

Nell'operazione di decodifica, la rappresentazione nello strato bottleneck h viene utilizzata per generare l'output $\hat{x}$, che viene mappato per ricostruire x[13], come mostrato nell'Equazione :

$$\hat{x} = f_2(w_j h + b_j)$$

dove f_2 è una funzione di attivazione per il decodificatore. w_j è la matrice dei pesi, b_j rappresenta un bias e $\hat{x}$ rappresenta un esempio di input ricostruito. Nota che w_j e b_j possono non essere correlati ai corrispondenti w_i e b_i per l'encoder.

5.1.3 Reconstruction Loss

In un modello autoencoder standard, viene calcolata una perdita di ricostruzione L per minimizzare la differenza tra l'output e l'input, come mostrato nell'Equazione (. Spesso è questa perdita di ricostruzione che viene utilizzata per compiti di rilevamento delle anomalie[13] .

$$L(x - \hat{x}) = \frac{1}{n} \sum_{t=1}^{n} |\hat{x}_t - x_t|$$

dove x rappresenta i dati di input, $\hat{x}$ indica i dati di output e n è il numero di campioni nel dataset di addestramento.

5.1.4 LSTM-Autoencoder

Un modello LSTM-Autoencoder sfrutta le capacità sia della rete neurale LSTM che dell'Autoencoder, costruendo le reti LSTM secondo gli schemi di encoder e decoder dell'Autoencoder. L'encoder ottiene la sequenza dei dati di input ad alta dimensione come un vettore di dimensione fissa. Utilizzando le celle di memoria delle LSTM, i dati elaborati dallo schema dell'encoder mantengono le dipendenze tra diversi punti dati all'interno di una sequenza temporale, mentre continua a ridurre la rappresentazione del vettore di input ad alta dimensione in una rappresentazione a bassa dimensione fino a raggiungere lo spazio latente[13].

Capitolo 6

Anomaly Detector

6.1 Architettura

Nella pratica, il rilevamento delle anomalie può essere implementato mediante l'uso di due tecniche differenti, la prima tecnica consta nel generare avvisi real time partendo dal sistema che si sta misurando, la seconda consiste nel registrare eventi per analisi successive. La prima metodologia esposta è un po' rischiosa poichè le anomalie non implicano necesariamente un problema nel sistema, tutto ciò porterebbe alla generazione di "rumore" e renderebbe il processo costoso. La seconda tecnica, anche se "risk-averse", non permetterebbe di intervenire in maniera tempestiva in caso di problemi nel sistema[11] . Per tali motivi nel seguente studio si è cercato di condurre un approccio ibrido, la cui logica è la seguente:

Nel capitolo 5 abbiamo detto che un autoencoder è un tipo di rete neurale che apprende a comprimere i dati di input in una rappresentazione di dimensione inferiore e poi a ricostruire l'input originale dai dati compressi quindi se i dati di input sono "normali", l'autoencoder sarà in grado di ricostruirli con un basso errore di ricostruzione ne segue che se i dati sono "anomali", l'errore di ricostruzione sarà significativamente più alto, è stata sfruttata questa logica per rilevare le anomalie, infatti dopo l'addestramento, si calcola la perdita di ricostruzione (MAE) sui dati di addestramento e di test, e sfruttando la distribuzione di probabilità della funzione di perdita sui dati di training si è stabilito un threshold, che se superato indica la presenza di un'anomalia.

6.2 Finestre temporali

Suddividere i dati in finestre temporali di dimensione fissa è un passaggio critico nella preparazione dei dati per modelli di machine learning che analizzano sequenze temporali, come per l'appunto le LSTM. Le finestre temporali permettono ai modelli di machine learning di analizzare i dati in modo sequenziale, catturando le dipendenze temporali tra i dati, inoltre aiutano a ridurre la complessità del problema segmentando i dati in blocchi più gestibili, perchè invece di analizzare un intero dataset in una volta, il modello può concentrarsi su segmenti di dimensioni fisse.[2] In questo studio, sono state impiegate più dimensioni di finestre temporali, empiricamente è stato determinato che la dimensione più efficiente ed efficace è quella di 100 valori per finestra.

Di seguito alleghiamo il codice impiegato per la realizzazione delle finestre.

```
def create_time_windows(data, window_size):
```

```
    windows = []
    for i in range(len(data) - window_size + 1):
        window = data[i:i+window_size]
        windows.append(window)
    return windows

window_size = 100

X_train = create_time_windows(X_train, window_size)
X_test = create_time_windows(X_test, window_size)
```

Listing 2: Funzione per la realizzazione di finestre temporali

6.3 Reshape dei dati per l'LSTM

L'operazione di *Reshape* è essenziale perchè, come ben sappiamo, le LSTM sono progettate per gestire dati sequenziali, dove ogni campione può avere una sequenza di passi temporali quindi il reshape organizza i dati in modo che ogni campione sia una sequenza di lunghezza fissa e ogni passo temporale contenga un insieme di features in questo caso due[3].

```
# Convert to numpy arrays
X_train = np.array(X_train)
X_test = np.array(X_test)

# Reshape inputs for LSTM [samples, timesteps, features]
X_train = X_train.reshape(X_train.shape[0], 100, X_train.shape
    [2])
X_test = X_test.reshape(X_test.shape[0], 100, X_test.shape[2])
```

Listing 3: Reshaping Data for LSTM

6.4 LSTM-Autoencoder impiegata

Dopo aver suddiviso i dati in finestre temporali, e aver applicato il reshaping, finalmente è stato possibile costruire il modello LSTM-Autoencoder:

```
def autoencoder_model(X):
    inputs = Input(shape=(X.shape[1], X.shape[2]))
```

```
L1 = LSTM(16, activation='tanh', return_sequences=True,
          kernel_regularizer=regularizers.l2(0.00))(inputs)
L2 = LSTM(8, activation='tanh', return_sequences=False)(L1)
L3 = RepeatVector(X.shape[1])(L2)
L4 = LSTM(8, activation='tanh', return_sequences=True)(L3)
L5 = LSTM(16, activation='tanh', return_sequences=True)(L4)
output = TimeDistributed(Dense(X.shape[2]))(L5)
model = Model(inputs=inputs, outputs=output)
return model
```

Listing 4: Definizione Autoencoder Model

La spiegazione del codice sopra è la seguente:

- **Input Layer**: Rappresenta l'input del modello con la forma `(batch_size, timesteps, features)`, dove `timesteps` rappresenta il numero di passi temporali e `features` in ciascun passo temporale.
- **Strati LSTM**:
  - **L1**: Un'unità LSTM con 16 neuroni, attivazione tangente iperbolica (`tanh`), che restituisce sequenze (`return_sequences=True`). Applica anche una regolarizzazione del kernel per limitare il sovradattamento.
  - **L2**: Un'unità LSTM con 8 neuroni, attivazione `tanh`, che restituisce solo l'output finale del passo temporale (`return_sequences=False`).
- **RepeatVector**: Replica il vettore di output di L2 per adattarsi alla forma richiesta dalle successive LSTM (`L3`).
- **Altri Strati LSTM**:
  - **L4** e **L5** sono strati LSTM simili a L1 e L2, ma invertiti per ripristinare la forma originale del dato di input.
- **Output Layer**: Uno strato `TimeDistributed` con un singolo strato denso (`Dense`) che restituisce un output con la stessa forma dell'input originale, per ogni passo temporale.
- **Modello**: Combinando tutti gli strati sopra definiti, viene creato un modello Keras `Model` con gli input e gli output specificati.

Layer (type)	Output Shape	Param #	Trainable params
input_2 (InputLayer)	(None, 100, 2)	0	0
lstm_4 (LSTM)	(None, 100, 16)	1216	1216
lstm_5 (LSTM)	(None, 8)	800	800
repeat_vector_1 (RepeatVector)	(None, 100, 8)	0	0
lstm_6 (LSTM)	(None, 100, 8)	544	544
lstm_7 (LSTM)	(None, 100, 16)	1600	1600
time_distributed_1 (TimeDistributed)	(None, 100, 2)	34	34

Tabella 1: Summary del modello.

6.4.1 Numero di epoche

Nel contesto del *Deep Learning* le *Epoche* rappresentano le iterazioni di addestramento del modello su tutto il set di dati di training, quindi durante un'epoca, il modello utilizza il set di addestramento per aggiornare i pesi dei suoi strati e cerca di minimizzare la funzione di loss definita, in questo caso il numero di epoche scelte, basandoci su risultati empirici è venti.

6.4.2 Batch size

La batch è un insieme di dati di addestramento utilizzati nel processo di training per calcolare il gradiente e aggiornare i pesi del modello, la dimensione della batch usata nello studio e di cinquanta.

6.4.3 Validation split

Il "validation split" valuta le prestazioni del modello durante l'addestramento stesso, e consiste nel dividere il dataset di addestramento in due parti: una parte, più piccola, viene usata come set di validazione per monitorare le prestazioni del modello durante l'addestramento, mentre la restante parte viene utilizzata per il training del modello. Nello studio la percentuale di dati destinati al set di validazione, dopo aver provato più percentuali, è del venticinque percento.

6.5 Loss di training

Tracciare e visualizzare la loss durante l'addestramento del modello risulta estremamente utile per svariati motivi:

1. **Monitoraggio delle prestazioni del modello:** La loss (l'errore assoluto medio, MAE) misura quanto il modello autoencoder stia apprendendo dai dati di adde-

stramento dunque monitorandola nel corso delle epoche, è possibile valutare se il modello stia migliorando, stagnando o peggiorando durante l'addestramento.

2. **Identificazione di overfitting o underfitting:** Osservando la differenza tra la loss sul set di addestramento e la loss sul set di validazione , è possibile determinare se il modello stia soffrendo di overfitting o underfitting, infatti avere un divario significativo tra *loss* e *val_loss* può indicare overfitting.

3. **Ottimizzazione dei parametri di addestramento:** Inoltre aiuta a ottimizzare i parametri di addestramento come il numero di epoche, il batch size o il tasso di apprendimento. Per esempio, se la val_loss inizia ad aumentare mentre la loss di addestramento continua a diminuire, potrebbe essere segno di overfitting, suggerendo di interrompere l'addestramento prima di compromettere le prestazioni complessive del modello.

Figura 11: Grafico della funzione di loss durante l'addestramento del modello

6.6 Loss distribution

Dopo aver addestrato il modello sui dati di training, è stato impiegato per effettuare predizioni sui dati di training stessi, in maniera tale da ottenere la distribuzione della loss media assoluta della serie temporale "sana", e fatto ciò è stato possibile ricavare il valore usato come threshold, tale valore è inteso come il valore più alto della funzione di loss.

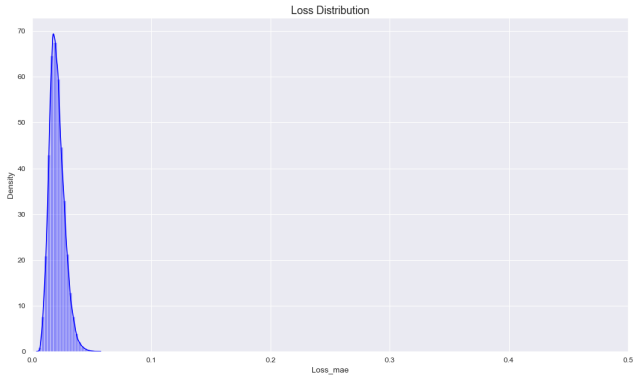


Figura 12: Grafico del Distribution plot della funzione di loss sui dati di training

6.7 Rilevare le anomalie

Infine si è passati al rilevamento delle anomalie vero e proprio, infatti calcolando la loss MAE sul set di test e visualizzando la distribuzione della loss, si sono potute evidenziare le anomalie rilevate rispetto alla soglia di threshold .

Per chiarezza è stato plottato un unico grafico della loss distribution dei dati di training e di test, con la soglia di threshold.

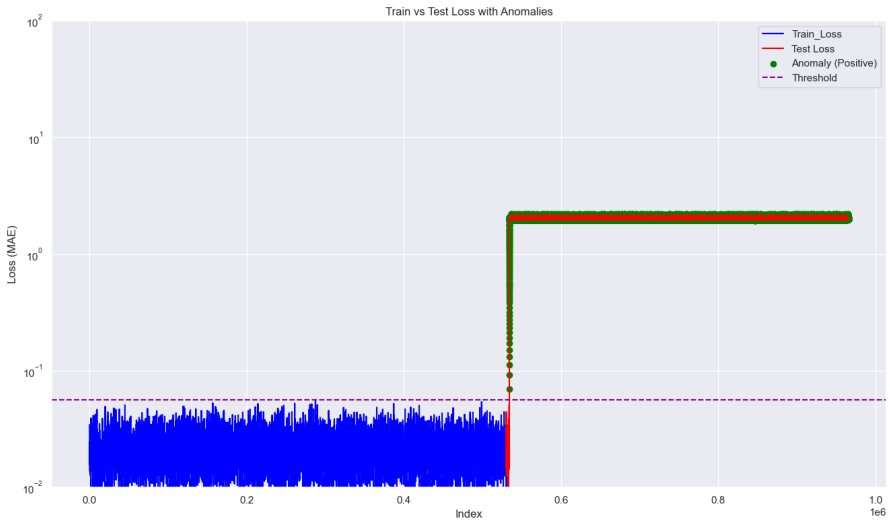


Figura 13: Grafico del Distribution plot dei dati di training e test con anomalie

Capitolo 7

Conclusioni

Lo studio sulla realizzazione dell'anomaly detector, realizzato tramite una rete ricorrente LSTM-Autoencoder, ha dimostrato la possibilità di rilevare anomalie nel funzionamento dell'impianto preso in esame con un'accuratezza del cento percento. Infatti calcolando metriche come *Precision*,*Recall* e *F1* siamo pervenuti ai seguenti risultati:

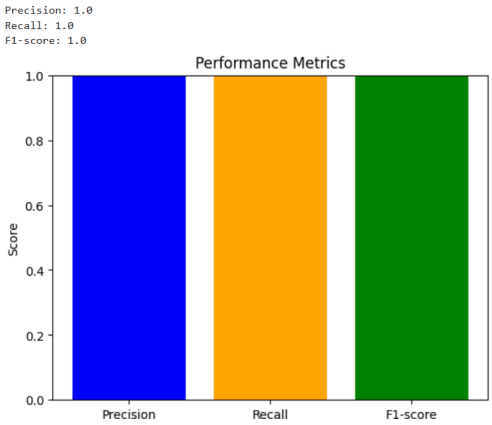


Figura 14: Metriche di performance

Tali metriche sono state ottenute calcolando *Precision*, *Recall* e *F1* sui dati della serie anomala , i quali sono stati etichettati come anomalie tranne le prime mille osservazioni che , si ricorda, erano state sostituite con le ultime mille della serie "sana", con i risultati predetti come anomali dall'Anomaly detector basandosi sui dati di test . Questi risultati indicano che il modello è stato in grado di identificare correttamente tutte le anomalie presenti nel dataset di test senza generare falsi positivi infatti la precisione del 100% assicura che ogni anomalia rilevata sia effettivamente tale, mentre il recall del 100% garantisce che tutte le anomalie presenti siano state rilevate dal modello.

L'F1-score, che rappresenta la media armonica tra precision e recall, conferma l'equilibrio perfetto tra queste due metriche, sottolineando la robustezza e l'affidabilità del modello.

Questi risultati sono particolarmente significativi considerando la complessità dei dati e la presenza di rumore dovuto a errori di setup dei sensori. La scelta di escludere le osservazioni problematiche dal dataset di training si è rivelata cruciale per migliorare le performance del modello, inoltre, la metodologia di sostituire le ultime mille osservazioni non anomale con le prime mille anomalie nel training set ha permesso di allenare il modello in maniera bilanciata, migliorando la sua capacità di distinguere tra dati normali e anomali.

In conclusione, l'anomaly detector sviluppato rappresenta uno strumento affidabile e preciso per il monitoraggio dei dati, in grado di rilevare anomalie con altissima accuratezza, questo modello può essere utilizzato in applicazioni reali per migliorare la qualità del monitoraggio dei sistemi e la gestione degli allarmi, riducendo al minimo i falsi positivi e assicurando una pronta identificazione delle anomalie reali.

Riferimenti bibliografici

[1] Shivanshu Aggarwal. *The Ultimate Guide to Building Your Own LSTM Models*. 2023. URL: https://www.projectpro.io/article/lstm-model/832.

[2] Jason Brownlee. *Deep Learning for Time Series Forecasting*. Machine Learning Mastery, 2018.

[3] Francois Chollet. *Deep learning with Python*. Simon e Schuster, 2021.

[4] Tom Dietterich. «Overfitting and undercomputing in machine learning». In: *ACM computing surveys (CSUR)* 27.3 (1995), pp. 326–327.

[5] Dario Bruneo Fabrizio De Vita e Sajal K. Das. «A Novel Data Collection Framework for Telemetry and Anomaly Detection in Industrial IoT Systems». In: *IEEE* (2020).

[6] Dario Floreano e Claudio Mattiussi. *Manuale sulle reti neurali*. Il mulino, 2002.

[7] Yoshua Bengio Ian Goodfellow e Aaron Courville. «Deep Learning». Book in preparation for MIT Press. 2016. URL: http://www.deeplearningbook.org.

[8] Scattolini R. Magni L. *Complementi di controlli automatici*. Pitagora Editrice, 2006.

[9] Kevin P Murphy. *Machine learning: a probabilistic perspective*. MIT press, 2012.

[10] Christopher Olah. *Understanding LSTM Networks*. 2015. URL: http://colah.github.io/posts/2015-08-Understanding-LSTMs/.

[11] Baron Schwartz Preetam Jinka. *Anomaly Detection for Monitoring A Statistical Approach to TimeSeries Anomaly Detection*. O'Reilly, 2015.

[12] Sebastian Ruder. «An overview of gradient descent optimization algorithms». In: *arXiv preprint arXiv:1609.04747* (2016).

[13] Yuanyuan Wei et al. *LSTM-Autoencoder based Anomaly Detection for Indoor Air Quality Time Series Data*. 2022. arXiv: 2204.06701 [cs.LG]. URL: https://arxiv.org/abs/2204.06701.